



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO De Tuxtepec

PROGRAMACION NATIVA PARA PLATAFORMAS MOVILES

REPORTE DE ACTIVIDADES

Nombre y Número de Control:

Cardoza Martínez Yasmin
22350393

DOCENTE:

JULIO AGUILAR CARMONA

CARRERA:

INGENIERÍA INFORMÁTICA

Semestre y grupo: 8A

San Juan Bautista Tuxtepec, Oaxaca.

12/ JUNIO/2026

INTRODUCCION

El desarrollo de aplicaciones móviles y multiplataforma ha evolucionado de manera significativa con la introducción de herramientas de código abierto que optimizan el rendimiento y la experiencia de usuario. En el marco de las actividades formativas de este periodo, el presente reporte documenta el proceso de aprendizaje y práctica en el desarrollo nativo utilizando Dart y Flutter. En primera instancia, se utilizaron entornos virtuales de ejecución rápida como DartPad para comprender la sintaxis. Posteriormente, los conocimientos teóricos y prácticos adquiridos se trasladaron al framework Flutter. En esta etapa, el enfoque se centró en la arquitectura basada en widgets, el diseño de interfaces de usuario dinámicas y adaptables, y la gestión de estados básicos para crear los cimientos de aplicaciones nativas de alto rendimiento.

ACTIVIDADES REALIZADAS

El primer ejercicio dentro de dart “Hola Mundo”

```
1  void main() {  
2      //var myName = 'Yasmin';  
3      //String myName = 'Yasmin';  
4      late final myName;  
5  
6      myName = 'Yasmin Cardoza';  
7  
8      print('Hola, ${myName.toUpperCase()}');  
9  }  
10
```

El propósito de este ejercicio es demostrar el uso de la palabra reservada `late` combinada con `final` para la declaración de variables cuya asignación de valor se pospone, así como el uso de funciones de manipulación de texto e interpolación dentro de una cadena.

Este ejercicio demuestra cómo Dart maneja la flexibilidad en el flujo de inicialización de datos sin sacrificar la seguridad del código (Null Safety) ni la inmutabilidad de los datos (`final`), optimizando el uso de memoria al evaluar variables solo cuando son requeridas.

Variables.dart

```
02_Variables.dart > main
Run | Debug
1 void main() {
2   final String pokemon = 'Ditto';
3   final int hp = 100;
4   bool? isAlive;
5   final List<String>abilities = ['impostor'];
6   final sprites = <String>['ditto/front.png', 'ditto/back.png'];
7
8   dynamic errorMessage = 'Hola';
9   errorMessage = true;
10  errorMessage = [1,2,3,4,5];
11  errorMessage = null;
12
13  print("""
14    $pokemon
15    $hp
16    $isAlive
17    $abilities
18    $sprites
19    $errorMessage
20    """);
21 }
```

El propósito de este ejercicio (guardado en el archivo 02_Variables.dart) es explorar la variedad de tipos de datos nativos que ofrece Dart. Se analiza el comportamiento de variables inmutables, el manejo de valores nulos (Null Safety), la estructura de colecciones (listas) y la flexibilidad del tipo especial dynamic.

Maps.dart

```
03_Maps.dart > main
Run | Debug
1 void main() {
2   final Map<String,dynamic> pokemon = {
3     'name': 'Ditto' ,
4     'hp': 100,
5     'isAlive': true,
6     'abilities' : <String> ['impostor'],
7     'sprites': <int, String>{
8       1: 'ditto/front.png' ,
9       2: 'ditto/back.png'
10    }
11  };
12
13  print(pokemon);
14 }
15
```

El propósito de este ejercicio es aprender a utilizar e implementar Mapas (Map) en Dart. Esta estructura permite almacenar datos organizados en pares de llave-valor, siendo fundamental para el manejo de respuestas JSON, configuraciones o el modelado de objetos complejos antes de convertirlos a clases. Esta práctica es clave para entender el flujo de datos en aplicaciones móviles reales. Se concluye que el correcto manejo de mapas anidados con tipado flexible permite estructurar objetos complejos de una manera limpia, eficiente y fuertemente controlada por el lenguaje.

ListIterableSet.dart

```
04_ListIterableSet.dart > main
Run | Debug
1 void main() {
2     final numbers = [1,2,3,4,5,6,7,8,9,10];
3
4     print ('Lista Original $numbers');
5     print ('longitud: ${numbers.length}');
6     print ('indice: ${numbers[0]}');
7     print ('Primero: ${numbers.first}');
8     print ('Ultimo: ${numbers.last}');
9     //Iterable
10    print ('Inversa: ${numbers.reversed}');
11
12    final numerosInvertidos= numbers.reversed;
13    print('Iterable: ${numerosInvertidos}');
14    print('Lista: ${numerosInvertidos.toList()}');
15    print('Set : ${numerosInvertidos.toSet()}');
16
17    final numerosMayoresA5 = numbers.where((int number) {
18        | return number > 5;
19    });
20
21    print ('>5: ${numerosMayoresA5.toSet()}');
22    }
23
24
```

En este ejercicio se analizan las propiedades nativas de las listas (List), comprender el comportamiento de los objetos de tipo Iterable mediante transformaciones lógicas (como la inversión de orden y el filtrado por predicciones), y aprender a convertir estas estructuras a colecciones de elementos únicos conocidas como Conjuntos (Set).

Al ejecutar `numbers.reversed`, Dart no devuelve otra lista de forma inmediata, sino un objeto de tipo Iterable. Un iterable es una colección de elementos que se pueden secuenciar, pero cuyos valores se evalúan de manera "perezosa" (lazy evaluation). En consola se distingue porque se encierra entre paréntesis (10, 9, 8, ...) en lugar de corchetes. Esta actividad fue para comprender el procesamiento de datos eficientes en Dart. Se concluye que el uso de métodos funcionales como `where` y propiedades como `.reversed` facilitan la manipulación de arreglos sin necesidad de implementar ciclos manuales (como `for` o `while`).

Clases.dart

```
06_Clases.dart > main
Run | Debug
1 void main() {
2   final wolverine = new Hero(name: "Logan", power: "Regeneracion");
3
4   print(wolverine);
5   print(wolverine.name);
6   print(wolverine.power);
7 }
8
9 class Hero {
10   String name;
11   String power;
12
13   Hero({required this.name, required this.power});
14
15   @override
16   String toString() {
17     return '$name - $power';
18   }
19 }
20
```

Esta práctica marca la base del desarrollo con Flutter al introducir el concepto de Clases. El ejercicio demuestra el modelado de un objeto de la vida real (en este caso, un héroe), la inicialización de propiedades por medio de constructores modernos y la sobrescritura de métodos heredados de la clase base Object.

Funciones.dart

```
05_Funciones.dart > main
Run | Debug
1 void main() {
2   print(greetEveryone());
3   print('Suma: ${addTwoNumbers(3,2)}');
4   print('Suma opcional: ${addTwoNumbersOptional(6)}');
5   print(greetPerson(name: 'Yasmin' ));
6 }
7
8 //Srting greetEveryone() {
9 // return 'Hello Everyone';
10 //}
11
12 String greetEveryone() => 'Hello everyone!';
13
14 int addTwoNumbers(int a, int b) => a + b;
15
16 int addTwoNumbersOptional(int a, [int b = 0]) {
17   //b= b ?? 0;
18   //b ??= 0;
19
20   return a + b;
21 }
22
23
24 String greetPerson({required String name, String message = 'Hola'}) {
25   return '$message $name';
26 }
```

El propósito de esta práctica es dominar los distintos tipos de funciones en Dart. Se analiza la diferencia entre la sintaxis tradicional de bloque de código y la sintaxis simplificada de flecha (Arrow Functions), además de profundizar en la flexibilidad de los parámetros de entrada: posicionales obligatorios, posicionales opcionales con valores por defecto y argumentos nombrados (required).

ConsultoresNombrados.dart

```
07_ConsultoresNombrados.dart > ...
2  void main() {
30
31    print(ironman);
32    print(spiderman);
33    print (wolverine);
34
35  }
36
37  class Hero{
38    String name;
39    String power;
40    bool isAlive;
41
42    Hero({
43      required this.name,
44      required this.power,
45      required this.isAlive
46    });
47
48    Hero.fromJson(Map<String, dynamic> json)
49      :name= json['name'],
50        power = json['power'],
51        isAlive = json['isAlive'];
52
53    @override
54    String toString() {
55      return '$name, $power - ${isAlive ? 'Yes :)' : 'Nope :('}';
56    }
57  }
```

```
07_ConsultoresNombrados.dart > ...
1
Run | Debug
2  void main() {
3  final Map<String, dynamic> spidermanJson ={
4    'name': 'Peter Parker',
5    'power': 'Aracnido',
6    'isAlive': true
7  };
8
9
10 final Map<String, dynamic> wolverineJson ={
11   'name': 'Logan',
12   'power': 'Regeneracion',
13   'isAlive': true
14 };
15
16 final wolverine = Hero.fromJson(wolverineJson);
17
18
19 final ironman = Hero(
20   name: 'Tony Stark',
21   power: 'Millonario',
22   isAlive: false
23 );
24
25 final spiderman = Hero(
26   name: spidermanJson['name'],
27   power: spidermanJson['power'],
28   isAlive: spidermanJson['isAlive'],
29 );
```

El propósito de este ejercicio es comprender e implementar Constructores Nombrados (Named Constructors) en Dart. Esta característica permite definir múltiples constructores para una misma clase con el fin de resolver diferentes escenarios de inicialización. El caso de uso principal demostrado es la deserialización de datos, es decir, transformar un mapa de información (Map<String, dynamic>) —que simula una respuesta de una API REST— en una instancia de objeto fuertemente tipada.

Inicializa el objeto utilizando argumentos nombrados obligatorios. Es útil cuando se crean instancias de manera manual escribiendo los datos directamente en el código fuente (como se observa con `ironman` en las líneas 19-23).

Esta actividad consolida el patrón de diseño fundamental para el consumo de servicios web en aplicaciones móviles. Se concluye que los constructores nombrados proveen una sintaxis limpia y mantenible para instanciar clases bajo diferentes contextos. La implementación de `.fromJson` reduce las líneas de código repetitivas en la capa de datos y asegura que los flujos asíncronos en Flutter puedan interpretar la información cruda del backend y transformarla inmediatamente en entidades lógicas de tipado seguro.

Gettersetting.dart

```
08_gettersetting.dart > main
Run | Debug
1 void main() {
2   final miCuadrado = Square(side: 5);
3
4
5   print('area: ${miCuadrado.area}');
6 }
7
8 class Square {
9   double _side;
10
11   Square ({required double side})
12     : assert(side >=0, 'Side must be >=0'),
13       _side = side;
14
15
16   set side(double value) {
17     print('Setting new value $value');
18     if (value < 0) throw 'Value must be >=0';
19
20     _side = value;
21   }
22
23
24   double get area {
25     return _side * _side;
26   }
27 }
28
```

El propósito de esta práctica es implementar los pilares del encapsulamiento y la validación de datos en las clases de Dart. Se modela una figura geométrica (Square) protegiendo sus propiedades internas contra asignaciones inválidas mediante el uso de atributos privados, aserciones en el constructor (assert), y controlando el flujo de lectura y escritura a través de los métodos especiales get y set.

Al ejecutar este archivo sin alterar los parámetros de la función principal, la consola imprime de forma limpia: área: 25.0

ClassesAbstractas.dart

```
09_ClassesAbstractas.dart > main
21  abstract class EnergyPlant {
31      void consumeEnergy(double amount);
32
33  }
34
35  class WindPlant extends EnergyPlant{
36      WindPlant({required double initialEnergy})
37          :super(energyLeft: initialEnergy, type: PlantType.wind);
38
39      @override
40      void consumeEnergy(double amount){
41          energyLeft -= amount;
42      }
43  }
44
45
46  class NuclearPlant implements EnergyPlant{
47      @override
48      double energyLeft;
49
50      @override
51      final PlantType type = PlantType.nuclear;
52
53      NuclearPlant({required this.energyLeft});
54
55      @override
56      void consumeEnergy(double amount){
57          energyLeft -= (amount*.5);
58      }
59  }
```

```
09_ClassesAbstractas.dart > main
Run | Debug
1  void main() {
2      final windPlant = WindPlant(
3          initialEnergy: 100,
4      );
5      final nuclearPlant = NuclearPlant(energyLeft: 1000);
6
7      print('Wind: ${chargePhone(windPlant)}');
8      print('Nuclear: ${chargePhone(nuclearPlant)}');
9
10 }
11
12 double chargePhone(EnergyPlant plant){
13     if(plant.energyLeft < 10){
14         throw Exception('Not enough energy');
15     }
16     return plant.energyLeft -10;
17 }
18
19 enum PlantType {nuclear, wind, water}
20
21 abstract class EnergyPlant {
22     double energyLeft;
23     final PlantType type;
24
25     EnergyPlant({
26         required this.energyLeft,
27         required this.type
28     });
29 }
```

El propósito de este ejercicio es comprender la utilidad de una Clase Abstracta como un molde o contrato arquitectónico de software. Se modela un sistema de plantas de energía (EnergyPlant) para demostrar el polimorfismo: cómo dos clases hijas (WindPlant y NuclearPlant) pueden adoptar la identidad de la clase padre pero reaccionar con comportamientos y reglas de negocio completamente distintas ante un mismo método. Al usar implements, la clase hija no hereda absolutamente nada del código o lógica del padre; únicamente adopta sus firmas como un contrato estricto. Por lo tanto, NuclearPlant se ve obligada a rediseñar y declarar explícitamente desde cero las propiedades `@override double energyLeft;` y `@override final PlantType type = ...` (líneas 47-51), así como su propio constructor independiente.

Al correr el programa principal, considerando los valores iniciales y la resta de 10 unidades que realiza la función `chargePhone`, la terminal arroja:

Wind: 90.0

Nuclear: 990.0

Mixin.dart

```
11_Mixin.dart > ...
26 class Pato extends Ave with Volador, Caminador, Nadador {}
27 class Tiburon extends Pez with Nadador {}
28 class PezVolador extends Pez with Nadador, Volador{}
29
Run | Debug
30 void main(){
31     final flipper = Delfin();
32     flipper.nadar();
33
34     final batman = Murcielago();
35     batman.volar();
36     batman.caminar();
37
38     final namor = Pato();
39     namor.nadar();
40     namor.volar();
41     namor.caminar();
42 }
```

```
11_Mixin.dart > ...
2  abstract class Animal {}
3
4  abstract class Mamifero extends Animal {}
5
6  abstract class Ave extends Animal {}
7
8  abstract class Pez extends Animal {}
9
10 mixin Volador {
11     void volar() => print('estoy volando');
12 }
13
14 mixin Caminador {
15     void caminar() => print('estoy caminando');
16 }
17
18 mixin Nadador {
19     void nadar() => print('estoy nadando');
20 }
21
22 class Delfin extends Mamifero with Nadador {}
23 class Murcielago extends Mamifero with Volador, Caminador {}
24 class Gato extends Mamifero with Caminador {}
25 class Paloma extends Ave with Volador, Caminador {}
26 class Pato extends Ave with Volador, Caminador, Nadador {}
27 class Tiburon extends Pez with Nadador {}
28 class PezVolador extends Pez with Nadador, Volador{}
29
Run | Debug
```

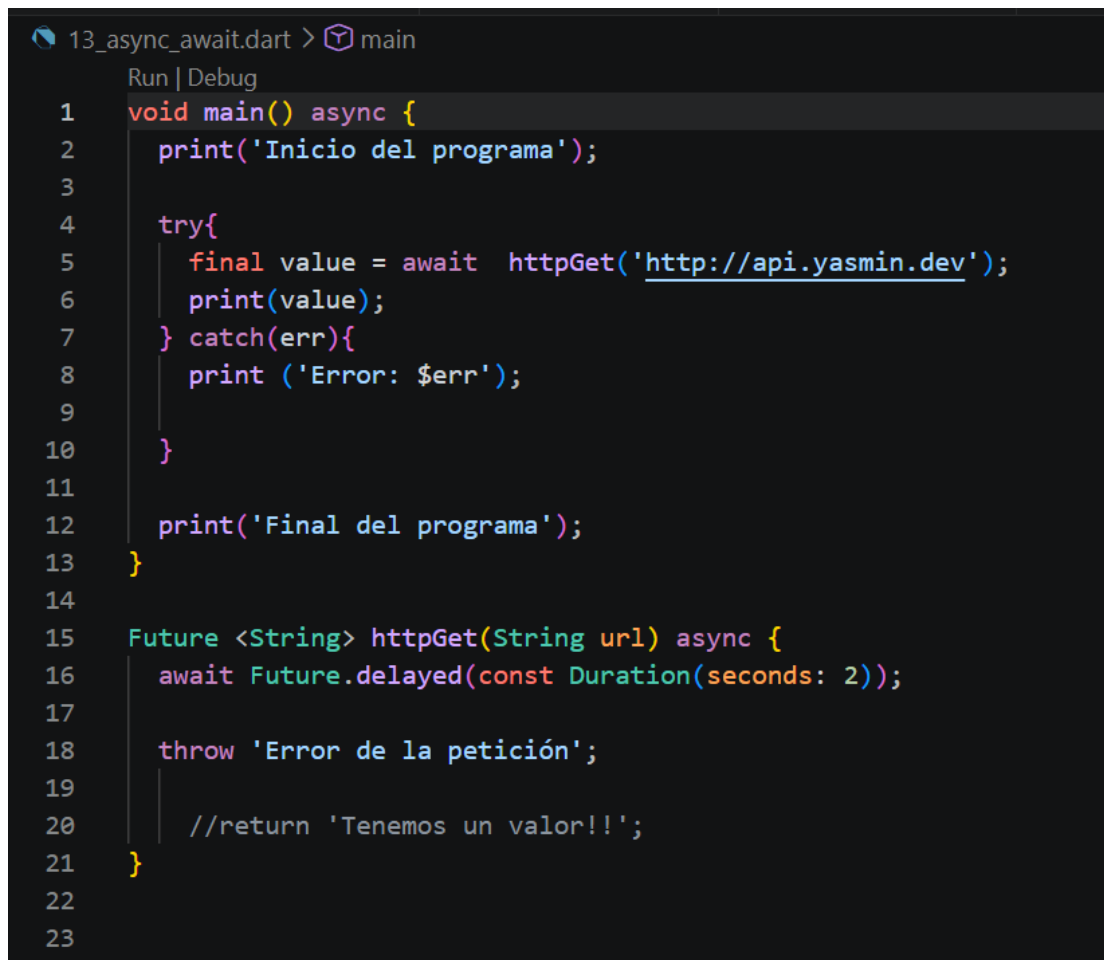
El propósito de este ejercicio es comprender e implementar el concepto de Mixins en Dart. A diferencia de las clases tradicionales o abstractas, los mixins permiten heredar o "inyectar" comportamientos y métodos a múltiples jerarquías de clases

sin establecer una relación rígida de padre e hijo. Esto soluciona de forma elegante el problema de la herencia múltiple, permitiendo que clases de distintos orígenes (como aves, mamíferos o peces) compartan capacidades comunes (como volar, caminar o nadar) de manera modular.

Al ejecutar por completo el archivo 11_Mixin.dart, la salida secuencial en la terminal es la siguiente:

- estoy nadando
- estoy volando
- estoy caminando
- estoy nadando
- estoy volando
- estoy caminando

Asyn_await.dart

A screenshot of an IDE window titled '13_async_await.dart > main'. The window has a 'Run | Debug' button. The code is written in Dart and uses the 'async-await' pattern. It starts with a 'void main() async {' block. Inside, it prints 'Inicio del programa', then enters a 'try{' block. In the try block, it calls 'await httpGet('http://api.yasmin.dev');' and prints the result. A 'catch(err){' block follows, printing 'Error: \$err'. After the try block, it prints 'Final del programa' and closes the main block with '}'. Below this, there is a 'Future <String> httpGet(String url) async {' function definition. It uses 'await Future.delayed(const Duration(seconds: 2));' to simulate a delay, then 'throw 'Error de la petición';' and a commented-out 'return 'Tenemos un valor!!';'. The function ends with '}'.

Esta actividad es un prerequisite fundamental para interactuar con arquitecturas reales en Flutter. En cualquier aplicación móvil moderna, las operaciones que involucran servicios externos deben ser asíncronas para evitar congelar la interfaz de usuario (UI), asegurando que las animaciones sigan corriendo fluidas mientras los datos se cargan tras bambalinas.

Se concluye que el patrón async-await proporciona una sintaxis limpia, legible y lineal (similar al código síncrono tradicional) que facilita la lectura del código, mientras que la implementación estricta de estructuras try-catch garantiza la resiliencia del software, permitiendo que la aplicación informe al usuario de un fallo de conexión en lugar de cerrarse de manera inesperada (crash).

Try_catch_finally.dart

```
14_try_catch_finally.dart > ...  
1  
Run | Debug  
2 void main() async {  
3   print('Inicio del programa');  
4  
5   try{  
6     final value = await httpGet('http://api.yasmin.dev');  
7     print(value);  
8   }on Exception catch(err){  
9     print('Tenemos una excepcion: $err');  
10  }catch(err) {  
11    print('OOPS!! Algo terrible ha pasado: $err');  
12  
13  } finally{  
14    print('Fin del try-catch');  
15  }  
16  
17  print('Final del programa');  
18 }  
19  
20 Future <String> httpGet(String url) async {  
21   await Future.delayed(const Duration(seconds: 2));  
22   throw Exception('No hay parametros en la URL');  
23  
24   //throw 'Error de la petición';  
25  
26   //return 'Tenemos un valor!!';  
27 }  
28
```

El propósito de esta práctica es implementar una estructura de control de errores de nivel de producción en Dart. Se explora cómo realizar un filtrado selectivo de excepciones utilizando la palabra reservada `on` para capturar fallos específicos del sistema, manteniendo un bloque `catch` genérico como respaldo, y se introduce el bloque `finally` para asegurar la ejecución de rutinas de limpieza de datos independientemente de si la operación asíncrona fue exitosa o fallida.

Al correr el programa principal, el orden cronológico de eventos en la terminal se despliega de la siguiente manera:

Inicio del programa

Tenemos una excepcion: Exception: No hay parametros en la URL

Fin del try-catch

Final del programa

Async.dart

A screenshot of a code editor with a dark theme. The title bar shows '15-async.dart > ...'. The code is written in Dart and uses syntax highlighting. Line numbers 1 through 16 are visible on the left. The code defines a main function that listens to a stream of numbers, doubling each value and delaying it by 1 second. The stream is created using an async* function that iterates over a list of initial values.

```
1
  Run | Debug
2  void main(){
3      emitNumbers().listen((value){
4          print('Stream value: $value');
5      });
6  }
7
8  Stream<int> emitNumbers() async*{
9      final initialValues = [1, 2, 3, 4, 5];
10
11      for(int i in initialValues){
12          i *=2; //i = i * 2
13          await Future.delayed(const Duration(seconds: 1));
14          yield i;
15      }
16  }
```

Esta actividad marca la cumbre del aprendizaje de Dart puro antes de adentrarse por completo en interfaces complejas de Flutter. Los Streams representan la columna vertebral de los patrones de arquitectura de estado y programación reactiva.

Se concluye que el dominio de las estructuras `async*` y `yield` dota al desarrollador de la capacidad para estructurar aplicaciones fluidas que consumen datos asíncronos concurrentes de alta frecuencia sin saturar la memoria del dispositivo, tales como sistemas de geolocalización por GPS en tiempo real, barras de progreso de descargas, o actualizaciones de bases de datos reactivas en la nube (como Firebase Firestore).

Stream.dart

```
16-stream.dart > emitNumbers
1
  Run | Debug
2 void main(){
3   emitNumbers().listen((value){
4     print('Stream value: $value');
5   });
6 }
7
8 Stream<int> emitNumbers(){
9   return Stream.periodic(const Duration(seconds: 1), (value){
10     return value;
11   }).take(10); // Stream.periodic
12 }
```

Dado que el contador implícito de `Stream.periodic` arranca desde cero y se detiene tras exactamente 10 emisiones debido al método `.take(10)`, los datos viajan segundo a segundo de la siguiente manera:

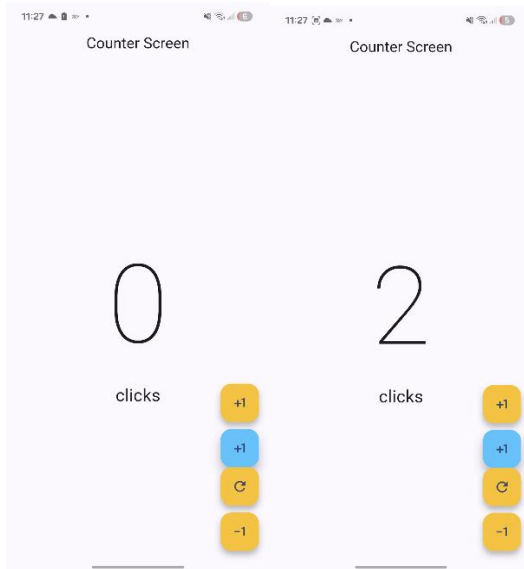
- **Segundo 1:** Primera ejecución del intervalo contador interno en 0
\$→ Stream value: 0
- **Segundo 2:** Segunda ejecución del intervalo contador interno en 1
\$→ Stream value: 1
- **Segundo 3:** Tercera ejecución del intervalo contador interno en 2
\$→ Stream value: 2
- *[... El proceso se repite sucesivamente con incrementos de una unidad por segundo ...]*
- **Segundo 10:** Décima ejecución del intervalo contador interno en 9 Stream value: 9. Al llegar a la cantidad límite estipulada por `.take(10)`, el Stream se cierra definitivamente de manera segura.

Stream value: 0
Stream value: 1
Stream value: 2
Stream value: 3
Stream value: 4
Stream value: 5
Stream value: 6
Stream value: 7
Stream value: 8
Stream value: 9

Se realizaron proyectos utilizando flutter, tales como “Prueba”, “yes_no_app”, “firts_flutter_project” y “flutter_jsonparsing”

Se anexan evidencias de las aplicaciones creadas.

Aplicación “Prueba”



El proyecto consiste en una aplicación móvil monopágina desarrollada en Flutter cuya función principal es actuar como un contador digital interactivo. Esta práctica sirve como introducción fundamental al manejo de la interfaz de usuario (UI) reactiva, el diseño de maquetación (Layouts) y la mutación de variables en tiempo de ejecución a través de componentes interactivos.

Aplicación “yes_no_app”



El propósito de esta práctica es diseñar y estructurar una pantalla de conversación en tiempo real. Este ejercicio permite dominar el manejo de listas dinámicas, la alineación condicional de componentes visuales (distinguiendo entre mensajes enviados y recibidos), y la integración de contenido multimedia (imágenes o GIFs) dentro de una burbuja de chat.

Scaffold: Actúa como el contenedor principal de la pantalla.

AppBar (Cabecera del Chat):

- **CircleAvatar:** Ubicado a la izquierda, muestra una imagen de perfil circular (en este caso, un avatar personalizado de un cerdito).
- **Text:** Muestra el nombre del contacto con el que se chatea ("My love").

ListView.builder: Es el componente más importante del cuerpo (body). En lugar de cargar todos los mensajes de golpe, este constructor optimiza la memoria renderizando en pantalla únicamente los mensajes que son visibles para el usuario mientras hacen *scroll*.

Burbujas de Chat Personalizadas (ChatBubble): Son contenedores estilizados (Container o DecoratedBox) con bordes redondeados (BorderRadius) que se repiten a lo largo de la lista.

CONCLUSIÓN

Hacer todas estas prácticas en clase me ayudó muchísimo a entender cómo se crea una aplicación móvil desde cero. Al principio, admito que no es fácil cambiar el chip, pero ver cómo todo va conectado paso a paso me dio una idea muy clara de cómo funcionan las plataformas actuales. Empezar con ejercicios sencillos me sirvió para perderle el miedo al código. Aprendí a cuidar cómo guardo la información, a ordenar las carpetas de mis archivos y a hacer que las funciones de mis programas hagan exactamente lo que necesito. También vi temas más avanzados como los procesos asíncronos y los streams, que básicamente sirven para que la aplicación pueda cargar datos pesados de internet, como mapas o descargas, sin que la pantalla del celular se trabe o se cierre a la fuerza.

Pasar todo lo que aprendí en teoría a las pantallas reales con Flutter fue la mejor parte. Con la aplicación del contador ("Prueba") entendí cómo un simple botón puede cambiar el número de la pantalla al instante. Y con la aplicación de chat ("yes_no_app"), aprendí a diseñar una interfaz real que acomoda los mensajes a la izquierda o derecha dependiendo de quién escriba, e incluso a meterle imágenes y GIFs.